



Qt Framework

C++ GUI Applications

- ❖ Qt is a cross-platform application framework developed in C++.
- ❖ It is supported on various platforms, including Windows, Linux, Mac OS, Android, iOS, Embedded Linux, QNX, VxWorks, Windows CE/RT, Integrity, Wayland, X11, Embedded Devices, and so on.
- ❖ Primarily used as a Human-Machine-Interface (HMI) or Graphical User Interface (GUI) framework. But it can be used to develop a command-line interface (CLI) applications
- ❖ QML (Qt Modeling Language) is a markup language like CSS and javascript to create GUI.
- ❖ Qt framework provides [some modules](#). [Qt Core](#) is one of them which has a lot of useful classes, such as [QObject](#), [QCoreApplication](#), [QDebug](#) and etc.
 - Almost every Qt application requires these modules.

- ❖ Qt classes inherit from QObject, and QObject offers event handling for Qt applications.
- ❖ **QDebug()** macro in QDebug from the QObject module has overloaded the cout stream operator of C++. This macro is used for debugging
- ❖ To create a project in the root of your project run: **qmake -project** (a .pro file is generated)
- ❖ To build the project, first make a directory, e.g., **build** and then in **build** run **qmake ..**
 - qmake command creates a Makefile. By running make, the project is built.
- ❖ **Qt Widgets** is a module that supports a lot of widgets, such as buttons, labels, edit, combo, list, dialog, and so on.
 - [QWidget](#) is the base class of all of the widgets.
- ❖ To create GUI applications you need to include at least
 - [QApplication](#) and QWidget.

```
#include <QDebug>

int main(void) {
    qDebug() << "Qt Version: " << QT_VERSION_STR;
    qDebug() << Qt::endl << "Hello Qt!" << Qt::endl;
    return 0;
}
```

- ❖ Every Qt GUI application must have exactly one instance of QApplication.
- ❖ QApplication provides support for command-line arguments for applications
 - In the .pro file you need add **QT += widgets** to link the widget shared object (the .so file) to the program
- ❖ GUI-based applications are event-driven, so they must respond to events (signals). The exec function starts the event loop, which ensures the application waits for user interactions until the user closes the window.
- ❖ User events will be received by the QApplication instance in an event queue(FIFO), which will then be notified to its child widgets
- ❖ Qt supports different layouts to arrange child widgets in a window

```
#include <QWidget>
#include <QApplication>

int main(int argc, char **argv) {
    QApplication app(argc, argv);

    QWidget window;
    window.setWindowTitle("Hello Qt!");
    window.show();

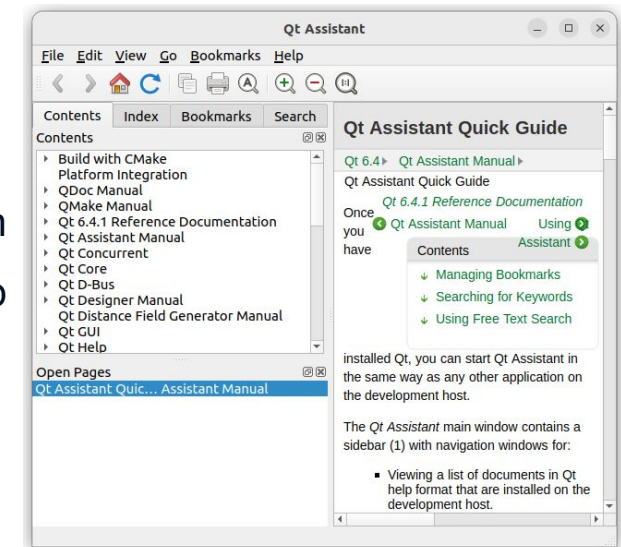
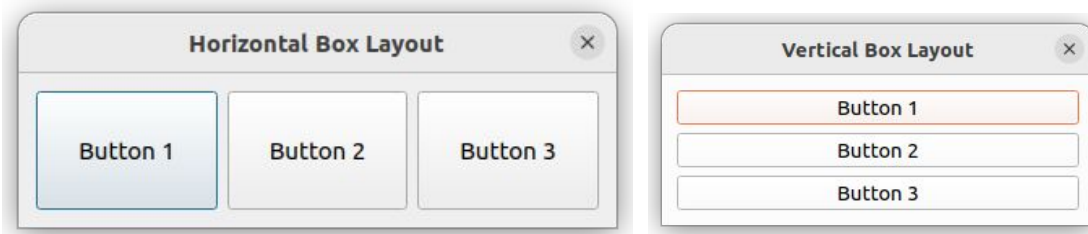
    return app.exec();
}
```

```
TEMPLATE = app
TARGET = example
INCLUDEPATH += .
```

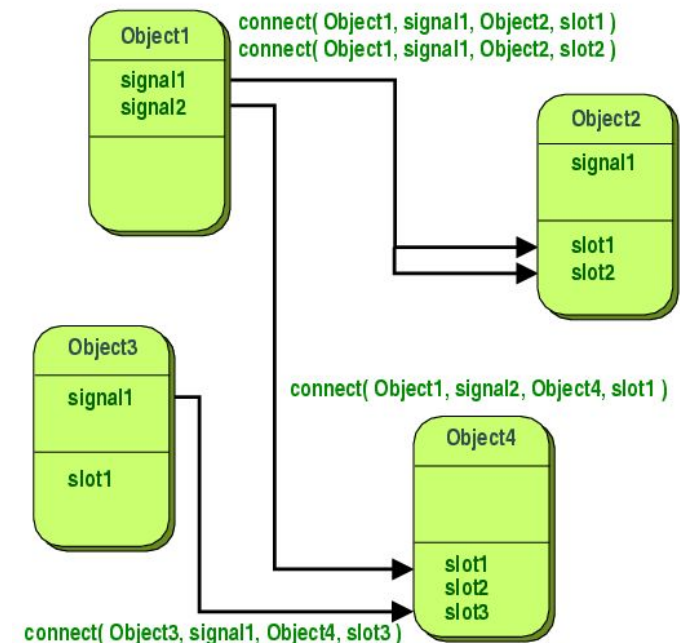
```
QT += widgets
```

```
# Input
SOURCES += main.cpp
```

- ❖ Qt supports horizontal, vertical box, box, grid and stacked layouts
 - QHBoxLayout and QVBoxLayout are two derived class from QBoxLayout
 - QHBoxLayout is QBoxLayout with QBoxLayout::Direction to QBoxLayout::LeftToRight
 - QVBoxLayout is QBoxLayout with QBoxLayout::Direction to QBoxLayout::TopToBottom
 - In addition to those values, QBoxLayout::Direction supports
 - QBoxLayout::LeftToRight: arranges the widgets from left to right
 - QBoxLayout::RightToLeft: arranges the widgets from right to left
 - QBoxLayout::TopToBottom: arranges the widgets from top to bottom
 - QBoxLayout::BottomToTop: arranges the widgets from bottom to top
- ❖ To use the Qt assistant, you can find it in ... /gcc_64/bin



- ❖ A GridLayout allows us to arrange widgets in a tabular fashion
- ❖ Specify the row and column where each widget must be added to the layout.
 - Even it is possible to specify the number of rows and columns a widget takes
- ❖ Signals and slots
 - Signals and slots are an integral part of the Qt Framework.
 - Qt signals are events, and slot functions are event handler functions.
 - Both signals and slots are normal C++ functions; only when they are marked as signals or slots, the Qt Framework will understand their purpose.
 - Every widget in Qt supports some signals and may also optionally support some slots.



- ❖ To connect a signal to a slot, the **connect** function is used.
- ❖ A stacked layout in an application that can have multiple windows; each window could be either a QWidget or QDialog and may have its own child widgets.
- ❖ There are more [layouts](#) like form layout and etc.
- ❖ If visual studio code cannot find the Qt headers
 - Open Settings: File > Preferences > Settings or Ctrl + ,
 - Search for “Include Path” and find **Default: Include Path**
 - Click on Add Item and the add **/usr/include/x86_64-linux-gnu/qt6/****

```
#ifndef DIALOG_H
#define DIALOG_H

#include <QDebug>
#include <QDialog>
#include <QHBoxLayout>
#include <QPushButton>

class dialog : public QDialog {
    QPushButton btnn;
    QHBoxLayout layout;
public:
    dialog();
private:
    void onButtonClicked();
};
#endif
```